

Stage 79 — per-moment thematic soundtrack for episode assembly

Five parts. Read this whole file before coding.

PART 1 — Fix MusicGen generation (lib/music.ts)

Currently `generateMusicTrack` uses `predictions.create({ model: "meta/musicgen" })`, which hits `POST /v1/models/meta/musicgen/predictions` and returns **404** (meta/musicgen is NOT an official model, cannot be run by name — confirmed). Switch to running BY VERSION.

- Add `const MUSICGEN_VERSION_ID` = the full hex id obtained via:

```
set -a; . /home/ubuntu/.prod.env; set +a; curl -s -H "Authorization: Bearer $REPLICATE_API_TOKEN" https://api.replicate.com/v1/models/meta/musicgen | python3 -c "import sys,json;print(json.load(sys.stdin)['latest_version']['id'])"
```

(it starts with `671ac645ce5e`). Paste the full id.

- `generateMusicTrack(mood)` : create via

```
getReplicate().predictions.create({ version: <resolved version>, input: {...} })
```

with the SAME input as now (prompt=MOOD_PROMPTS[mood], duration=MUSIC_TRACK_SECONDS, model_version:"stereo-large", output_format:"mp3", normalization_strategy:"peak").

- If create throws an error whose message mentions "version" and one of 404 / "not found" / "Invalid version" / "does not exist", fetch the current version ONCE via

```
getReplicate().models.get("meta","musicgen") -> latest_version.id
```

, then retry create.

- Add helper `resolveMusicgenVersion()` caching the version in a module variable (seeded with `MUSICGEN_VERSION_ID`). Do NOT change polling (`getPredictionState`) or `getOrCreateMusicTrack` S3-cache logic.

PART 2 — Scene-based music plan (NEW file lib/music-plan.ts; pure + one LLM fn)

- Export type `SceneMoodInput = { index: number; action?: string; dialogue?: string; kind?: string } .`
- `buildMusicPlan(scenes, meta:{title?;logline?;synopsis?})` : use the SAME gpt-4o LLM call that `pickMood` in `lib/music.ts` uses (import the same client/helper) to return, per scene, `{ mood: Mood | "none", intensity: number 0..1 }` as STRICT JSON. On LLM failure fall back to a single `pickMood` mood for all scenes with intensity 0.6.
- PURE `mergeMoodSegments(perScene)` : group CONSECUTIVE scenes with the same mood into `{ mood: Mood; startSceneIndex; endSceneIndex; intensity }` (intensity = average over the segment's scenes). Scenes whose mood is "none" belong to NO segment (silence there).
- PURE `limitMoods(segments, max=3)` : if unique moods > max, keep the max most frequent by scene coverage and recolor the other segments to the nearest remaining mood by frequency. Never drop segments.
- PURE `toTimelineSegments(segments, seamOffsets: number[], totalDuration: number)` : seamOffsets are

the cut positions in the OUTPUT timeline (scene boundaries). Scene i spans

```
[ (i==0?0:seamOffsets[i-1]) .. (seamOffsets[i] ?? totalDuration) ] . Return
{ mood, startSec, endSec, intensity } where a segment covering scenes a..b has startSec =
start of
scene a and endSec = end of scene b.
```

- `summarizePlan(...)` -> Russian string, e.g. “напряжённая → мистическая (2 сегмента, 1 сцена без музыки)”.

PART 3 — Timeline music overlay (lib/ffmpeg.ts)

- Add to `AssembleOptions` :

```
resolveMusicSegments?(workDir, seamOffsets: number[], totalDuration: number) => Promise<Array<{ path: string; startSec: number; endSec: number; intensity: number }> | null> .
```

 If set and it returns a non-empty array, use the segmented mix; otherwise keep the existing single `resolveMusic` path (backward compatibility is REQUIRED).
- PURE builder `buildMusicSegmentsMixFilter({ segments, hasVoice, totalDuration })` -> filter-graph
 string: per segment, input is looped, `atrim` to segment length, `afade` in/out 1s, `volume` = the LINEAR value of $(-18 \text{ dB} * \text{intensity})$, `adelay` `startSec*1000` on both channels. Then

```
musicMixed = amix of all segments; if hasVoice:
ducked = sidechaincompress(musicMixed keyed by
voice, threshold~0.03, ratio 8, attack 20, release 300) , out = amix(voice, ducked) ; if no
voice:
out = musicMixed .
```
- Update `buildFinalRenderArgs` to emit one `-stream_loop -1 -i <path>` per segment plus `-filter_complex` and `-map` of the final audio label when segments are present. Do NOT break the single-track path.
- Any mix failure → fallback render WITHOUT music (like the existing “music mix failed” catch); assembly always completes; set `musicApplied` correctly.

PART 4 — Assembly integration (lib/assemble.ts)

- Instead of one mood: build scene inputs from the already-loaded scenes (use available Scene fields
 such as action/dialogue/videoPrompt/description + order), call

```
buildMusicPlan → mergeMoodSegments → limitMoods(..., 3) .
```
- Implement `resolveMusicSegments` : for each UNIQUE mood among the segments call

```
getOrCreateMusicTrack(projectId, mood)
```

 in parallel (Promise.all; cache is reused), download one file per mood into `workDir`; use `toTimelineSegments(segments, seamOffsets, totalDuration)` for the windows; return `{ path (that mood's file), startSec, endSec, intensity }[]` . Pass `seamOffsets` and `totalDuration` from the stitch graph into the callback.
- Keep the “music” progress stage at 30–40% (do not change the budget). Do not touch the smooth 0–100% progress with elapsed time.
- Add to `episode_stitch` `resultData` : `musicPlan` (segments + count of scenes without music), `musicSummary` (`summarizePlan` output), `musicError` (on failure). note : if music ends up absent due to an error → “Музыка недоступна — собрано без музыки” (keep existing note behavior).

PART 5 — UI (app/project/[id]/episode/[episodeId]/episode-view.tsx, assembly block ~line 1507)

Replace the static text “Фоновая музыка подбирается по настроению серии автоматически; если музыка

недоступна, эпизод собирается без неё.” with dynamic text from the last assembly’s resultData:

- `musicApplied` + `musicSummary` → “Музыка: {musicSummary}”
- `musicError` → “Музыка недоступна: {musicError}”
- no assembly yet → short hint that music is auto-picked per the episode’s moments and, if unavailable, the episode is assembled without it.

Read from the already-pollled stitch job resultData; add minimal fields to the result type as needed.

TESTS — scripts/test-stage79.ts (pure functions only, NO network, NO generation)

Same style as scripts/test-stage78.ts (passed/total counter, non-zero exit on failure):

- `mergeMoodSegments`: consecutive same-mood merge; “none” excluded; mixed sequences.
- `limitMoods`: >3 moods reduced to 3 without losing segments; ≤3 unchanged.
- `toTimelineSegments`: correct startSec/endSec for a multi-scene segment; for the first and last scene.
- `buildMusicSegmentsMixFilter`: string contains atrim / afade / adelay / volume / sidechaincompress / amix when hasVoice; no sidechaincompress when there is no voice.
- `summarizePlan`: format.
- `MUSICGEN_VERSION_ID` is a non-empty hex; `generateMusicTrack` source runs by version.

Any URL literal in the test must be built by concatenation (“http”+“s://”).